

Guia Completo: Angular e TypeScript do Zero

Integração com API Spring Boot - Projeto Envio API

Índice

1. Fundamentos de TypeScript
 2. Fundamentos de Angular
 3. Estrutura de um Projeto Angular
 4. Criando seu Primeiro Componente
 5. Formulários Reativos
 6. HTTP e Serviços
 7. Rotas e Navegação
 8. Autenticação e JWT
 9. Prática Completa
-

Parte 1: Fundamentos de TypeScript

O que é TypeScript?

TypeScript é JavaScript com tipos. Ele adiciona tipagem estática ao JavaScript, permitindo que você encontre erros antes mesmo de executar o código.

Conceitos Básicos de TypeScript

1. Variáveis e Tipos

JavaScript (sem tipos):

```
let nome = "João";  
let idade = 25;
```

TypeScript (com tipos explícitos):

```
let nome: string = "João";  
let idade: number = 25;  
let ativo: boolean = true;
```

Tipos básicos:

- `string`: texto

- **number**: números
- **boolean**: true/false
- **any**: qualquer tipo (evite quando possível)

2. Arrays e Objetos

```
// Array de strings
let frutas: string[] = ["maçã", "banana", "laranja"];

// Array de números
let numeros: number[] = [1, 2, 3, 4, 5];

// Objeto com tipos definidos
let pessoa: {
  nome: string;
  idade: number;
  email: string;
} = {
  nome: "João",
  idade: 25,
  email: "joao@email.com"
};
```

3. Interfaces (Contratos de Tipos)

```
// Define a "forma" de um objeto
interface Usuario {
  id: number;
  nome: string;
  email: string;
  ativo: boolean;
}

// Agora posso usar essa interface
let usuario: Usuario = {
  id: 1,
  nome: "João",
  email: "joao@email.com",
  ativo: true
};

// Função que recebe um Usuario
function exibirUsuario(user: Usuario): void {
  console.log(`${user.nome} - ${user.email}`);
}
```

4. Classes

```
class Carro {
  // Propriedades
  marca: string;
  modelo: string;
  ano: number;

  // Construtor (executado quando cria o objeto)
  constructor(marca: string, modelo: string, ano: number) {
    this.marca = marca;
    this.modelo = modelo;
    this.ano = ano;
  }

  // Método
  exibirInfo(): string {
    return `${this.marca} ${this.modelo} (${this.ano})`;
  }
}

// Criar um objeto (instância)
let meuCarro = new Carro("Toyota", "Corolla", 2023);
console.log(meuCarro.exibirInfo()); // "Toyota Corolla (2023)"
```

5. Funções e Arrow Functions

```
// Função tradicional
function somar(a: number, b: number): number {
  return a + b;
}

// Arrow function (mais moderna)
const somar = (a: number, b: number): number => {
  return a + b;
};

// Arrow function simplificada (uma linha)
const somar = (a: number, b: number): number => a + b;

// Uso
let resultado = somar(5, 3); // 8
```

6. Optional e Null

```
// ? significa que a propriedade é opcional
interface Produto {
  id: number;
  nome: string;
  preco?: number; // Opcional
}
```

```
}

let produto1: Produto = {
  id: 1,
  nome: "Notebook"
  // preco não precisa ser informado
};

let produto2: Produto = {
  id: 2,
  nome: "Mouse",
  preco: 50.00
};
```

Parte 2: Fundamentos de Angular

O que é Angular?

Angular é um framework frontend desenvolvido pelo Google para criar aplicações web modernas. Ele organiza o código em componentes, módulos e serviços.

Arquitetura Básica

```
Angular App
├── Componentes (o que o usuário vê)
├── Serviços (lógica de negócio)
├── Módulos (organização)
└── Rotas (navegação)
```

Conceitos Principais

1. Componentes

Um componente é uma unidade que combina HTML, CSS e TypeScript.

Estrutura de um componente:

```
envio-list/
├── envio-list.component.ts    (lógica)
├── envio-list.component.html  (visual)
└── envio-list.component.css   (estilo)
```

Exemplo simples:

```
// envio-list.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-envio-list', // Nome da tag HTML
  templateUrl: './envio-list.component.html',
  styleUrls: ['./envio-list.component.css']
})
export class EnvioListComponent {
  // Propriedades (variáveis)
  titulo: string = "Lista de Envios";
  envios: string[] = ["Envio 1", "Envio 2", "Envio 3"];

  // Métodos (funções)
  adicionarEnvio(): void {
    this.envios.push("Novo Envio");
  }
}
```

```
<!-- envio-list.component.html -->
<div>
  <h1>{{ titulo }}</h1>

  <ul>
    <li *ngFor="let envio of envios">
      {{ envio }}
    </li>
  </ul>

  <button (click)="adicionarEnvio()">Adicionar</button>
</div>
```

Explicação:

- `{{ titulo }}`: interpolação (exibe o valor)
- `*ngFor`: loop (repete para cada item)
- `(click)`: evento (chama função ao clicar)

2. Data Binding (Ligação de Dados)

```
// component.ts
export class MeuComponent {
  nome: string = "João";
  idade: number = 25;
}
```

```
<!-- Interpolação: exibe valor -->
<p>Nome: {{ nome }}</p>
<p>Idade: {{ idade }}</p>

<!-- Property Binding: define propriedade -->
<input [value]="nome">
<img [src]="urlDaImagem">

<!-- Event Binding: escuta eventos -->
<button (click)="salvar()">Salvar</button>
<input (input)="onInputChange($event)">

<!-- Two-way Binding: ida e volta -->
<input [(ngModel)]="nome">
```

3. Diretivas

Diretivas estruturais (alteram o DOM):

```
<!-- *ngIf: mostra/esconde -->
<div *ngIf="usuarioLogado">
  Bem-vindo!
</div>

<!-- *ngFor: repete -->
<ul>
  <li *ngFor="let item of lista">
    {{ item }}
  </li>
</ul>

<!-- *ngSwitch: múltiplas condições -->
<div [ngSwitch]="status">
  <p *ngSwitchCase="'ativo'">Ativo</p>
  <p *ngSwitchCase="'inativo'">Inativo</p>
  <p *ngSwitchDefault>Desconhecido</p>
</div>
```

Diretivas de atributo (alteram comportamento):

```
<!-- [ngClass]: adiciona classes CSS -->
<button [ngClass]="{'ativo': isActive, 'desabilitado': isDisabled}">
  Clique
</button>

<!-- [ngStyle]: define estilos inline -->
<div [ngStyle]="{'color': corTexto, 'font-size': tamanhoFonte + 'px'}">
```

```
    Texto  
  </div>
```

4. Serviços

Serviços contêm lógica de negócio e comunicação com APIs.

```
// envio.service.ts  
import { Injectable } from '@angular/core';  
  
@Injectable({  
  providedIn: 'root' // Disponível em toda aplicação  
})  
export class EnvioService {  
  
  // Método que retorna dados  
  obterEnvios(): string[] {  
    return ["Envio 1", "Envio 2", "Envio 3"];  
  }  
  
  // Método que recebe parâmetro  
  buscarPorId(id: number): string {  
    return `Envio ${id}`;  
  }  
}
```

Usando o serviço no componente:

```
// component.ts  
import { Component } from '@angular/core';  
import { EnvioService } from './envio.service';  
  
export class MeuComponent {  
  envios: string[] = [];  
  
  // Injeção de dependência (Angular fornece automaticamente)  
  constructor(private envioService: EnvioService) {}  
  
  ngOnInit(): void {  
    // Carrega dados quando componente é criado  
    this.envios = this.envioService.obterEnvios();  
  }  
}
```

5. Ciclo de Vida do Componente

```
import { Component, OnInit, OnDestroy } from '@angular/core';

export class MeuComponent implements OnInit, OnDestroy {

  // Executado quando componente é criado
  ngOnInit(): void {
    console.log("Componente criado!");
    // Carregar dados aqui
  }

  // Executado quando componente é destruído
  ngOnDestroy(): void {
    console.log("Componente destruído!");
    // Limpar recursos aqui
  }
}
```

Ordem dos hooks:

1. `constructor` - primeiro
2. `ngOnInit` - após criar
3. `ngOnDestroy` - ao destruir

Parte 3: Estrutura de um Projeto Angular

Criando o Projeto

```
# 1. Instalar Angular CLI globalmente
npm install -g @angular/cli

# 2. Criar novo projeto
ng new envio-frontend

# Perguntas que aparecerão:
# - Adicionar roteamento? (Y) Sim
# - Qual formato de stylesheet? (CSS) - escolha CSS para começar

# 3. Entrar na pasta
cd envio-frontend

# 4. Rodar o projeto
ng serve

# Abre em http://localhost:4200
```

Estrutura de Pastas Explicada


```
envio-frontend/  
├── src/  
│   ├── app/                # Código da aplicação  
│   │   ├── app.component.ts # Componente principal  
│   │   ├── app.component.html # HTML principal  
│   │   └── app.module.ts     # Módulo principal  
│   ├── assets/             # Imagens, ícones, etc.  
│   ├── environments/       # Configurações (dev/prod)  
│   ├── index.html          # HTML raiz  
│   └── styles.css           # Estilos globais  
├── angular.json             # Configuração do projeto  
├── package.json             # Dependências  
└── tsconfig.json            # Configuração TypeScript
```

O que é cada arquivo?

app.module.ts - Módulo Principal

```
import { NgModule } from '@angular/core';  
import { BrowserModule } from '@angular/platform-browser';  
  
import { AppComponent } from './app.component';  
  
@NgModule({  
  declarations: [  
    AppComponent // Componentes que pertencem a este módulo  
  ],  
  imports: [  
    BrowserModule // Módulos que este módulo usa  
  ],  
  providers: [], // Serviços disponíveis  
  bootstrap: [AppComponent] // Componente inicial  
})  
export class AppModule { }
```

app.component.ts - Componente Raiz

```
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-root', // <app-root></app-root> no index.html  
  templateUrl: './app.component.html',  
  styleUrls: ['./app.component.css']  
})  
export class AppComponent {  
  title = 'envio-frontend';  
}
```

index.html - HTML Raiz

```
<!doctype html>
<html lang="pt-BR">
<head>
  <meta charset="utf-8">
  <title>Envio Frontend</title>
</head>
<body>
  <!-- Angular renderiza tudo aqui -->
  <app-root></app-root>
</body>
</html>
```

Parte 4: Criando seu Primeiro Componente

Passo a Passo

1. Gerar Componente

```
ng generate component components/envio-list
# ou forma curta:
ng g c components/envio-list
```

Cria automaticamente:

- envio-list.component.ts
- envio-list.component.html
- envio-list.component.css
- Atualiza app.module.ts

2. Editar o Componente TypeScript

```
// components/envio-list/envio-list.component.ts
import { Component, OnInit } from '@angular/core';

@Component({
  selector: 'app-envio-list',
  templateUrl: './envio-list.component.html',
  styleUrls: ['./envio-list.component.css']
})
export class EnvioListComponent implements OnInit {

  // Propriedades
  titulo: string = "Lista de Envios";
```

```
envios: any[] = []; // Array vazio inicialmente
carregando: boolean = false;

// Construtor (executado primeiro)
constructor() {
  console.log("Componente criado!");
}

// Executado após construtor
ngOnInit(): void {
  console.log("Componente inicializado!");
  this.carregarEnvios();
}

// Método personalizado
carregarEnvios(): void {
  this.carregando = true;

  // Simula carregamento (depois substituiremos por chamada HTTP)
  setTimeout(() => {
    this.envios = [
      { id: 1, nome: "Envio 1", cep: "12345678" },
      { id: 2, nome: "Envio 2", cep: "87654321" }
    ];
    this.carregando = false;
  }, 1000);
}

deletar(id: number): void {
  if (confirm("Deseja deletar?")) {
    this.envios = this.envios.filter(e => e.id !== id);
  }
}
}
```

3. Editar o Template HTML

```
<!-- components/envio-list/envio-list.component.html -->
<div class="container">
  <h1>{{ titulo }}</h1>

  <!-- Mostra "Carregando..." enquanto carregando é true -->
  <div *ngIf="carregando">
    <p>Carregando envios...</p>
  </div>

  <!-- Mostra lista quando não está carregando -->
  <div *ngIf="!carregando">
    <!-- Se não tem envios -->
    <p *ngIf="envios.length === 0">Nenhum envio encontrado</p>
```

```
<!-- Se tem envios, mostra tabela -->
<table *ngIf="envios.length > 0" class="table">
  <thead>
    <tr>
      <th>ID</th>
      <th>Nome</th>
      <th>CEP</th>
      <th>Ações</th>
    </tr>
  </thead>
  <tbody>
    <!-- Repete para cada envio -->
    <tr *ngFor="let envio of envios">
      <td>{{ envio.id }}</td>
      <td>{{ envio.nome }}</td>
      <td>{{ envio.cep }}</td>
      <td>
        <button (click)="deletar(envio.id)" class="btn btn-danger">
          Deletar
        </button>
      </td>
    </tr>
  </tbody>
</table>
</div>
</div>
```

4. Usar o Componente

```
<!-- app.component.html -->
<app-envio-list></app-envio-list>
```

Parte 5: Formulários Reativos

Por que usar Formulários Reativos?

- Validação mais fácil
- Melhor controle do estado
- Testes mais simples

Passo a Passo

1. Importar Módulo

```
// app.module.ts
import { ReactiveFormsModule } from '@angular/forms';
```

```
@NgModule({
  imports: [
    BrowserModule,
    ReactiveFormsModule // Adicionar aqui
  ]
})
```

2. Criar Formulário no Componente

```
// envio-form.component.ts
import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-envio-form',
  templateUrl: './envio-form.component.html'
})
export class EnvioFormComponent implements OnInit {

  // Formulário reativo
  envioForm: FormGroup;

  // Mensagem de erro
  mensagemErro: string = '';

  // Injeção do FormBuilder
  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    // Criar formulário com validações
    this.envioForm = this.fb.group({
      nomeRemetente: ['', [Validators.required, Validators.minLength(3)]],
      endereco: ['', Validators.required],
      cepOrigem: ['', [Validators.required, Validators.pattern(/^\d{8}$/)]],
      cepDestino: ['', [Validators.required, Validators.pattern(/^\d{8}$/)]],
      larguraCaixa: [0, [Validators.required, Validators.min(0.1)]],
      alturaCaixa: [0, [Validators.required, Validators.min(0.1)]],
      comprimentoCaixa: [0, [Validators.required, Validators.min(0.1)]],
      peso: [0, [Validators.required, Validators.min(0.1)]]
    });
  }

  // Método chamado ao submeter
  onSubmit(): void {
    if (this.envioForm.valid) {
      console.log("Formulário válido!", this.envioForm.value);
      // Aqui você chamaria o serviço para salvar
    } else {
      this.mensagemErro = "Preencha todos os campos corretamente!";
    }
  }
}
```

```
}

// Método auxiliar para facilitar acesso aos campos
get f() {
  return this.envioForm.controls;
}
}
```

3. Template HTML do Formulário

```
<!-- envio-form.component.html -->
<form [formGroup]="envioForm" (ngSubmit)="onSubmit()">

  <!-- Nome Remetente -->
  <div class="form-group">
    <label>Nome do Remetente:</label>
    <input
      type="text"
      formControlName="nomeRemetente"
      class="form-control"
      [class.is-invalid]="f.nomeRemetente.invalid && f.nomeRemetente.touched">

    <!-- Mensagens de erro -->
    <div *ngIf="f.nomeRemetente.invalid && f.nomeRemetente.touched"
      class="invalid-feedback">
      <div *ngIf="f.nomeRemetente.errors?.['required']">
        Nome é obrigatório
      </div>
      <div *ngIf="f.nomeRemetente.errors?.['minlength']">
        Nome deve ter no mínimo 3 caracteres
      </div>
    </div>
  </div>

  <!-- CEP Origem -->
  <div class="form-group">
    <label>CEP Origem:</label>
    <input
      type="text"
      formControlName="cepOrigem"
      class="form-control"
      placeholder="12345678"
      maxlength="8">

    <div *ngIf="f.cepOrigem.invalid && f.cepOrigem.touched" class="invalid-
feedback">
      <div *ngIf="f.cepOrigem.errors?.['required']">
        CEP é obrigatório
      </div>
      <div *ngIf="f.cepOrigem.errors?.['pattern']">
        CEP deve ter 8 dígitos
```

```
        </div>
      </div>
    </div>

    <!-- Botão Submit -->
    <button
      type="submit"
      [disabled]="envioForm.invalid"
      class="btn btn-primary">
      Criar Envio
    </button>

    <!-- Mensagem de erro geral -->
    <div *ngIf="mensagemErro" class="alert alert-danger">
      {{ mensagemErro }}
    </div>
  </form>
```

Explicação das validações:

- `Validators.required`: campo obrigatório
- `Validators.minLength(3)`: mínimo de caracteres
- `Validators.pattern(/^\d{8}$/)`: regex (8 dígitos)
- `Validators.min(0.1)`: valor mínimo

Parte 6: HTTP e Serviços

Como Fazer Requisições HTTP

1. Importar HttpClientModule

```
// app.module.ts
import { HttpClientModule } from '@angular/common/http';

@NgModule({
  imports: [
    BrowserModule,
    HttpClientModule // Adicionar aqui
  ]
})
```

2. Criar Serviço HTTP

```
// services/envio.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
```

```
import { Observable } from 'rxjs';

// Interface para tipar os dados
export interface Envio {
  envioId?: number;
  nomeRemetente: string;
  endereco: string;
  cepOrigem: string;
  cepDestino: string;
  larguraCaixa: number;
  alturaCaixa: number;
  comprimentoCaixa: number;
  peso: number;
}

@Injectable({
  providedIn: 'root'
})
export class EnvioService {

  // URL da API (ajuste para sua porta)
  private apiUrl = 'http://localhost:8080/api';

  // Injetar HttpClient
  constructor(private http: HttpClient) {}

  // GET: Buscar todos
  listar(): Observable<Envio[]> {
    return this.http.get<Envio[]>(`${this.apiUrl}/envios`);
  }

  // GET: Buscar por ID
  buscarPorId(id: number): Observable<Envio> {
    return this.http.get<Envio>(`${this.apiUrl}/envios/${id}`);
  }

  // POST: Criar novo
  criar(envio: Envio): Observable<any> {
    return this.http.post(`${this.apiUrl}/envios`, envio);
  }

  // DELETE: Deletar
  deletar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/envios/${id}`);
  }

  // PATCH: Atualizar parcial
  atualizarParcial(id: number, dados: Partial<Envio>): Observable<void> {
    return this.http.patch<void>(`${this.apiUrl}/envios/${id}`, dados);
  }
}
```


3. Usar o Serviço no Componente

```
// envio-list.component.ts
import { Component, OnInit } from '@angular/core';
import { EnvioService, Envio } from '../services/envio.service';

@Component({
  selector: 'app-envio-list',
  templateUrl: './envio-list.component.html'
})
export class EnvioListComponent implements OnInit {

  envios: Envio[] = [];
  carregando: boolean = false;
  erro: string = '';

  // Injetar serviço
  constructor(private envioService: EnvioService) {}

  ngOnInit(): void {
    this.carregarEnvios();
  }

  carregarEnvios(): void {
    this.carregando = true;
    this.erro = '';

    // Subscribe = "inscrever-se" para receber os dados
    this.envioService.listar().subscribe({
      // next = quando dados chegam com sucesso
      next: (envios) => {
        this.envios = envios;
        this.carregando = false;
      },
      // error = quando ocorre erro
      error: (err) => {
        this.erro = 'Erro ao carregar envios';
        this.carregando = false;
        console.error(err);
      }
    });
  }

  deletar(id: number): void {
    if (confirm('Deseja realmente deletar?')) {
      this.envioService.deletar(id).subscribe({
        next: () => {
          // Recarrega a lista após deletar
          this.carregarEnvios();
        },
        error: (err) => {
          alert('Erro ao deletar');
        }
      });
    }
  }
}
```

```
        console.error(err);
      }
    });
  }
}
```

O que é Observable?

Observable representa um fluxo de dados assíncrono. Você se inscreve com `subscribe()` para receber os valores.

```
// Observable emite valores ao longo do tempo
this.envioService.listar()
  .subscribe({
    next: (dados) => {
      // Recebe os dados aqui
      console.log(dados);
    },
    error: (erro) => {
      // Recebe erro aqui
      console.error(erro);
    },
    complete: () => {
      // Executado quando completa (opcional)
      console.log('Completo!');
    }
  });
```

Parte 7: Rotas e Navegação

Configurar Rotas

1. Criar Módulo de Rotas

```
// app-routing.module.ts
import { NgModule } from '@angular/core';
import { RouterModule, Routes } from '@angular/router';
import { EnvioListComponent } from '../components/envio-list/envio-list.component';
import { EnvioFormComponent } from '../components/envio-form/envio-form.component';
import { LoginComponent } from '../components/login/login.component';

// Definir rotas
const routes: Routes = [
  { path: 'login', component: LoginComponent },
  { path: 'envios', component: EnvioListComponent },
];
```

```
{ path: 'envios/novo', component: EnvioFormComponent },
{ path: '', redirectTo: '/envios', pathMatch: 'full' }, // Rota padrão
{ path: '**', redirectTo: '/envios' } // Rota não encontrada
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule]
})
export class AppRoutingModule { }
```

2. Importar no Módulo Principal

```
// app.module.ts
import { AppRoutingModule } from './app-routing.module';

@NgModule({
  imports: [
    BrowserModule,
    AppRoutingModule // Adicionar aqui
  ]
})
```

3. Adicionar router-outlet

```
<!-- app.component.html -->
<nav>
  <a routerLink="/envios">Lista</a>
  <a routerLink="/envios/novo">Novo Envio</a>
  <a routerLink="/login">Login</a>
</nav>

<!-- Aqui o Angular renderiza o componente da rota ativa -->
<router-outlet></router-outlet>
```

4. Navegar Programaticamente

```
// component.ts
import { Router } from '@angular/router';

export class MeuComponent {
  constructor(private router: Router) {}

  irParaLista(): void {
    this.router.navigate(['/envios']);
  }
}
```

```
irParaNovo(): void {  
  this.router.navigate(['/envios/novo']);  
}  
}
```

Parte 8: Autenticação e JWT

Fluxo de Autenticação

1. Usuário faz login
2. Backend retorna JWT token
3. Token é salvo no localStorage
4. Token é enviado em todas as requisições

Implementação

1. Serviço de Autenticação

```
// services/auth.service.ts  
import { Injectable } from '@angular/core';  
import { HttpClient } from '@angular/common/http';  
import { Observable, BehaviorSubject } from 'rxjs';  
import { tap } from 'rxjs/operators';  
  
export interface LoginRequest {  
  login: string;  
  senha: string;  
}  
  
export interface LoginResponse {  
  token: string;  
  login: string;  
}  
  
@Injectable({  
  providedIn: 'root'  
})  
export class AuthService {  
  private apiUrl = 'http://localhost:8080/auth';  
  
  // BehaviorSubject = Observable que guarda o último valor  
  private tokenSubject = new BehaviorSubject<string | null>(  
    localStorage.getItem('token') // Pega token salvo  
  );  
  
  // Observable público (outros componentes podem observar)  
  public token$ = this.tokenSubject.asObservable();
```

```

    constructor(private http: HttpClient) {}

    login(credentials: LoginRequest): Observable<LoginResponse> {
        return this.http.post<LoginResponse>(`${this.apiUrl}/login`, credentials)
            .pipe(
                tap(response => {
                    // Salva token no localStorage
                    localStorage.setItem('token', response.token);
                    localStorage.setItem('login', response.login);
                    // Atualiza o BehaviorSubject
                    this.tokenSubject.next(response.token);
                })
            );
    }

    logout(): void {
        localStorage.removeItem('token');
        localStorage.removeItem('login');
        this.tokenSubject.next(null);
    }

    getToken(): string | null {
        return localStorage.getItem('token');
    }

    isAuthenticated(): boolean {
        return !!this.getToken(); // !! converte para boolean
    }
}

```

2. Interceptor (Adiciona Token Automaticamente)

```

// interceptors/auth.interceptor.ts
import { Injectable } from '@angular/core';
import {
    HttpRequest,
    HttpHandler,
    HttpEvent,
    HttpInterceptor
} from '@angular/common/http';
import { Observable } from 'rxjs';
import { AuthService } from '../services/auth.service';

@Injectable()
export class AuthInterceptor implements HttpInterceptor {

    constructor(private authService: AuthService) {}

    intercept(request: HttpRequest<any>, next: HttpHandler):
    Observable<HttpEvent<any>> {

```

```
    const token = this.authService.getToken();

    // Se tem token, adiciona no header
    if (token) {
        const cloned = request.clone({
            headers: request.headers.set('Authorization', `Bearer ${token}`)
        });
        return next.handle(cloned);
    }

    // Se não tem token, passa a requisição sem modificar
    return next.handle(request);
}
```

3. Registrar Interceptor

```
// app.module.ts
import { HTTP_INTERCEPTORS } from '@angular/common/http';
import { AuthInterceptor } from '../interceptors/auth.interceptor';

@NgModule({
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true // Permite múltiplos interceptors
    }
  ]
})
```

4. Guard (Protege Rotas)

```
// guards/auth.guard.ts
import { Injectable } from '@angular/core';
import { CanActivate, Router } from '@angular/router';
import { AuthService } from '../services/auth.service';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {

  constructor(
    private authService: AuthService,
    private router: Router
  ) {}

  canActivate(): boolean {
```

```
    if (this.authService.isAuthenticated()) {
      return true; // Permite acesso
    } else {
      this.router.navigate(['/login']); // Redireciona para login
      return false; // Bloqueia acesso
    }
  }
}
```

5. Usar Guard nas Rotas

```
// app-routing.module.ts
import { AuthGuard } from './guards/auth.guard';

const routes: Routes = [
  { path: 'login', component: LoginComponent },
  {
    path: 'envios',
    component: EnvioListComponent,
    canActivate: [AuthGuard] // Protege a rota
  }
];
```

6. Componente de Login

```
// components/login/login.component.ts
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

@Component({
  selector: 'app-login',
  templateUrl: './login.component.html',
  styleUrls: ['./login.component.css']
})
export class LoginComponent {
  loginForm: FormGroup;
  errorMessage: string = '';

  constructor(
    private fb: FormBuilder,
    private authService: AuthService,
    private router: Router
  ) {
    this.loginForm = this.fb.group({
      login: ['', [Validators.required]],
      senha: ['', [Validators.required]]
    });
  }
```

```

    }

    onSubmit() {
      if (this.loginForm.valid) {
        this.authService.login(this.loginForm.value).subscribe({
          next: () => {
            this.router.navigate(['/envios']);
          },
          error: (err) => {
            this.errorMessage = 'Login ou senha inválidos';
            console.error(err);
          }
        });
      }
    }
  }
}

```

```

<!-- components/login/login.component.html -->
<div class="login-container">
  <h2>Login</h2>

  <form [formGroup]="loginForm" (ngSubmit)="onSubmit()">
    <div class="form-group">
      <label>Login:</label>
      <input type="text" formControlName="login" class="form-control">
      <div *ngIf="loginForm.get('login')?.hasError('required') &&
loginForm.get('login')?.touched">
        Login é obrigatório
      </div>
    </div>

    <div class="form-group">
      <label>Senha:</label>
      <input type="password" formControlName="senha" class="form-control">
      <div *ngIf="loginForm.get('senha')?.hasError('required') &&
loginForm.get('senha')?.touched">
        Senha é obrigatória
      </div>
    </div>

    <div *ngIf="errorMessage" class="alert alert-danger">
      {{ errorMessage }}
    </div>

    <button type="submit" [disabled]="loginForm.invalid" class="btn btn-primary">
      Entrar
    </button>
  </form>
</div>

```


Parte 9: Prática Completa - Projeto Passo a Passo

Passo 1: Criar Projeto

```
ng new envio-frontend
cd envio-frontend
ng serve
```

Passo 2: Criar Estrutura de Pastas

```
# Criar componentes
ng g c components/login
ng g c components/envio-list
ng g c components/envio-form

# Criar serviços
ng g s services/auth
ng g s services/envio

# Criar guards
ng g g guards/auth

# Criar interceptors
ng g interceptor interceptors/auth
```

Passo 3: Configurar CORS no Spring Boot

Adicione no `SecurityConfig.java`:

```
@Bean
public CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOrigins(List.of("http://localhost:4200"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "PATCH",
"DELETE", "OPTIONS"));
    configuration.setAllowedHeaders(List.of("*"));
    configuration.setAllowCredentials(true);

    UrlBasedCorsConfigurationSource source = new
UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}
```

E no `securityFilterChain`:

```
.cors(cors -> cors.configurationSource(corsConfigurationSource()))
```

Passo 4: Criar Interfaces/Models

```
// models/envio.model.ts
export interface EnvioRequest {
  nomeRemetente: string;
  endereco: string;
  cepOrigem: string;
  cepDestino: string;
  larguraCaixa: number;
  alturaCaixa: number;
  comprimentoCaixa: number;
  peso: number;
}

export interface FreteResponse {
  valorPAC: string;
  prazoPAC: string;
  mensagemPAC: string;
  valorSEDEX: string;
  prazoSEDEX: string;
  mensagemSEDEX: string;
}

export interface EnvioDetalhe {
  envioId: number;
  nomeRemetente: string;
  endereco: string;
  cepOrigem: string;
  cepDestino: string;
  larguraCaixa: number;
  comprimentoCaixa: number;
  alturaCaixa: number;
  peso: number;
  frete?: FreteResponse;
  mensagemGeral?: string;
}

export interface EnvioFreteResponse {
  nomeRemetente: string;
  cepOrigem: string;
  cepDestino: string;
  frete: FreteResponse;
  mensagem: string;
}
```

Passo 5: Implementar Serviços Completos

Serviço de Envios

```
// services/envio.service.ts
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { EnvioRequest, EnvioDetalhe, EnvioFreteResponse } from
'../models/envio.model';

@Injectable({
  providedIn: 'root'
})
export class EnvioService {
  private apiUrl = 'http://localhost:8080/api';

  constructor(private http: HttpClient) {}

  listar(): Observable<EnvioDetalhe[]> {
    return this.http.get<EnvioDetalhe[]>(`${this.apiUrl}/envios`);
  }

  buscarPorId(id: number): Observable<EnvioDetalhe> {
    return this.http.get<EnvioDetalhe>(`${this.apiUrl}/envios/${id}`);
  }

  criar(envio: EnvioRequest): Observable<EnvioFreteResponse> {
    return this.http.post<EnvioFreteResponse>(`${this.apiUrl}/envios`, envio);
  }

  atualizarParcial(id: number, campos: Partial<EnvioRequest>): Observable<void> {
    return this.http.patch<void>(`${this.apiUrl}/envios/${id}`, campos);
  }

  deletar(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/envios/${id}`);
  }
}
```

Passo 6: Componente de Listagem Completo

```
// components/envio-list/envio-list.component.ts
import { Component, OnInit } from '@angular/core';
import { EnvioService } from '../../../services/envio.service';
import { EnvioDetalhe } from '../../../models/envio.model';

@Component({
  selector: 'app-envio-list',
  templateUrl: './envio-list.component.html',
  styleUrls: ['./envio-list.component.css']
})
```

```

export class EnvioListComponent implements OnInit {
  envios: EnvioDetalhe[] = [];
  loading = false;
  error: string = '';

  constructor(private envioService: EnvioService) {}

  ngOnInit(): void {
    this.carregarEnvios();
  }

  carregarEnvios(): void {
    this.loading = true;
    this.envioService.listar().subscribe({
      next: (envios) => {
        this.envios = envios;
        this.loading = false;
      },
      error: (err) => {
        this.error = 'Erro ao carregar envios';
        this.loading = false;
        console.error(err);
      }
    });
  }

  deletar(id: number): void {
    if (confirm('Deseja realmente deletar este envio?')) {
      this.envioService.deletar(id).subscribe({
        next: () => {
          this.carregarEnvios(); // Recarrega a lista
        },
        error: (err) => {
          alert('Erro ao deletar envio');
          console.error(err);
        }
      });
    }
  }
}

```

```

<!-- components/envio-list/envio-list.component.html -->
<div class="envio-list-container">
  <h2>Lista de Envios</h2>

  <div *ngIf="loading" class="loading">
    Carregando...
  </div>

  <div *ngIf="error" class="alert alert-danger">
    {{ error }}
  </div>
</div>

```

```

</div>

<table class="table" *ngIf="!loading && envios.length > 0">
  <thead>
    <tr>
      <th>ID</th>
      <th>Remetente</th>
      <th>CEP Origem</th>
      <th>CEP Destino</th>
      <th>Frete PAC</th>
      <th>Frete SEDEX</th>
      <th>Ações</th>
    </tr>
  </thead>
  <tbody>
    <tr *ngFor="let envio of envios">
      <td>{{ envio.envioId }}</td>
      <td>{{ envio.nomeRemetente }}</td>
      <td>{{ envio.cepOrigem }}</td>
      <td>{{ envio.cepDestino }}</td>
      <td>
        <span *ngIf="envio.frete">
          R$ {{ envio.frete.valorPAC }} - {{ envio.frete.prazoPAC }} dias
        </span>
        <span *ngIf="!envio.frete">-</span>
      </td>
      <td>
        <span *ngIf="envio.frete">
          R$ {{ envio.frete.valorSEDEX }} - {{ envio.frete.prazoSEDEX }} dias
        </span>
        <span *ngIf="!envio.frete">-</span>
      </td>
      <td>
        <button (click)="deletar(envio.envioId)" class="btn btn-danger">
          Deletar
        </button>
      </td>
    </tr>
  </tbody>
</table>

<div *ngIf="!loading && envios.length === 0" class="empty-state">
  Nenhum envio cadastrado
</div>
</div>

```

Passo 7: Componente de Formulário Completo

```

// components/envio-form/envio-form.component.ts
import { Component } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';

```

```
import { Router } from '@angular/router';
import { EnvioService } from '../../../services/envio.service';
import { EnvioRequest } from '../../../models/envio.model';

@Component({
  selector: 'app-envio-form',
  templateUrl: './envio-form.component.html',
  styleUrls: ['./envio-form.component.css']
})
export class EnvioFormComponent {
  envioForm: FormGroup;
  loading = false;
  error: string = '';

  constructor(
    private fb: FormBuilder,
    private envioService: EnvioService,
    private router: Router
  ) {
    this.envioForm = this.fb.group({
      nomeRemetente: ['', [Validators.required]],
      endereco: ['', [Validators.required]],
      cepOrigem: ['', [Validators.required, Validators.pattern(/^\d{8}$/)]],
      cepDestino: ['', [Validators.required, Validators.pattern(/^\d{8}$/)]],
      larguraCaixa: [0, [Validators.required, Validators.min(0.1)]],
      alturaCaixa: [0, [Validators.required, Validators.min(0.1)]],
      comprimentoCaixa: [0, [Validators.required, Validators.min(0.1)]],
      peso: [0, [Validators.required, Validators.min(0.1)]]
    });
  }

  onSubmit() {
    if (this.envioForm.valid) {
      this.loading = true;
      const envio: EnvioRequest = this.envioForm.value;

      this.envioService.criar(envio).subscribe({
        next: (response) => {
          alert(`Envio criado! Frete PAC: R$ ${response.frete.valorPAC}`);
          this.router.navigate(['/envios']);
        },
        error: (err) => {
          this.error = 'Erro ao criar envio. Verifique os dados.';
          this.loading = false;
          console.error(err);
        }
      });
    }
  }

  get f() {
    return this.envioForm.controls;
  }
}
```

Passo 8: Configurar App Module Completo

```
// app.module.ts
import { NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { HttpClientModule, HTTP_INTERCEPTORS } from '@angular/common/http';
import { FormsModule, ReactiveFormsModule } from '@angular/forms';

import { AppComponent } from './app.component';
import { LoginComponent } from './components/login/login.component';
import { EnvioListComponent } from './components/envio-list/envio-list.component';
import { EnvioFormComponent } from './components/envio-form/envio-form.component';
import { AppRoutingModuleModule } from './app-routing.module';
import { AuthInterceptor } from './interceptors/auth.interceptor';

@NgModule({
  declarations: [
    AppComponent,
    LoginComponent,
    EnvioListComponent,
    EnvioFormComponent
  ],
  imports: [
    BrowserModule,
    HttpClientModule,
    FormsModule,
    ReactiveFormsModule,
    AppRoutingModuleModule
  ],
  providers: [
    {
      provide: HTTP_INTERCEPTORS,
      useClass: AuthInterceptor,
      multi: true
    }
  ],
  bootstrap: [AppComponent]
})
export class AppModule { }
```

Fluxo Completo de Funcionamento

1. **Usuário acessa** `/login`
2. **Faz login** → recebe JWT token
3. **Token é salvo** no `localStorage`
4. **Interceptor adiciona token** em todas as requisições

5. **Usuário acessa** `/envios` (protegido por AuthGuard)
6. **Componente chama** `EnvioService.listar()`
7. **Serviço faz requisição HTTP** com token no header
8. **Spring Boot valida token** e retorna dados
9. **Angular exibe os dados** na tela

Comandos Úteis

```
# Criar novo componente
ng generate component components/envio-list

# Criar novo serviço
ng generate service services/envio

# Criar novo guard
ng generate guard guards/auth

# Rodar aplicação (porta 4200)
ng serve

# Build para produção
ng build --prod
```

Dicas Importantes

1. **Sempre use TypeScript** (tipagem forte)
2. **Use Observables** para operações assíncronas
3. **Separe lógica de negócio** em serviços
4. **Use Guards** para proteger rotas
5. **Use Interceptors** para adicionar headers automaticamente
6. **Valide formulários** com Reactive Forms

Dicas Finais para Aprendizado

1. **Pratique:** Crie pequenos projetos para fixar o conhecimento
2. **Leia erros:** A mensagem de erro geralmente indica exatamente o problema
3. **Use console.log:** Para debugar e entender o fluxo
4. **Documentação:** <https://angular.io/docs>
5. **Stack Overflow:** Para dúvidas específicas

Próximos Passos

1. Aprender RxJS (operadores como `map`, `filter`, `catchError`)
 2. Aprender testes (Jasmine/Karma)
 3. Aprender Angular Material (componentes prontos)
 4. Aprender lazy loading (carregar módulos sob demanda)
-

Conclusão

Este guia cobre todos os fundamentos necessários para começar com Angular e TypeScript. Pratique cada conceito criando pequenos projetos e, gradualmente, você estará pronto para criar aplicações completas!

Boa sorte no seu aprendizado! 🚀

Recursos Adicionais

- **Documentação oficial Angular:** <https://angular.io/docs>
 - **Documentação TypeScript:** <https://www.typescriptlang.org/docs/>
 - **RxJS:** <https://rxjs.dev/>
 - **Angular Material:** <https://material.angular.io/>
 - **Stack Overflow:** <https://stackoverflow.com/questions/tagged/angular>
-

Documento criado para o projeto Envio API - Integração Angular + Spring Boot